CampusConnect
Campus Event and Club Management Portal

WEB TECHNOLOGY PROJECT REPORT
V SEMESTER BTech [PROGRAMME NAME]

FLEXICORE-3131

SUBMITTED TO

SCHOOL OF COMPUTER ENGINEERING
M.I.T., MANIPAL



By

[STUDENT 1 NAME]
Registration No. [STUDENT 1 REGISTRATION NUMBER]

[STUDENT 2 NAME]
Registration No. [STUDENT 2 REGISTRATION NUMBER]

Under the guidance of: [FACULTY NAME]

[SUBMISSION MONTH AND YEAR]



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent institution of MAHE, Manipal)

Team Details and Work Split

Sl.	Name	Reg. No.	Two primary assigned modules	Signature
1	[STUDENT 1 NAME]	[STUDENT 1 REGISTRATION NUMBER]	1. Authentication & Student Dashboard 2. Event Discovery & Registration	[SIGNATURE 1]
2	[STUDENT 2 NAME]	[STUDENT 2 REGISTRATION NUMBER]	1. Organizer Event Management 2. Organizer Dashboard & Attendees	[SIGNATURE 2]

Allocation note: these are assigned review, maintenance and demonstration responsibilities, not a claim of individual implementation history. The initial application and report were generated with Codex assistance. Both students must review, adapt and acknowledge their actual contributions. Shared work: architecture, schema, API integration, common design, integration testing, documentation and final report review.

1. Project Overview

CampusConnect is a working campus event portal with a React interface, Express REST backend and persistent SQLite database. Students create accounts, discover events, reserve one seat and manage their own registrations. Organizers create and maintain their own events and view confirmed attendees. Six focused features demonstrate the complete browser-to-database flow without external services.

1.1 Purpose of the Website

The portal gives fictional campus events one searchable home. It helps a student find an activity, understand its time and venue, and confirm a place. A personal dashboard makes cancellation clear. Organizers maintain accurate information and check who is attending. The project is intended for a reliable classroom demonstration rather than a public institutional deployment.

1.2 Objectives

1. Student registration, login and logout.
2. Database-driven event discovery with search and category filters.
3. Event details and capacity-safe registration.
4. Private student dashboard and cancellation.
5. Authorized organizer event creation, editing and deletion.
6. Organizer statistics and private confirmed-attendee lists.

1.3 Target Users

Students use discovery and personal registrations; authorized organizers manage events. Mobile users receive responsive cards and forms. Semantic landmarks, explicit labels, keyboard-accessible links and buttons, visible focus, a skip link, reduced-motion support and announced error/status messages improve accessibility. No formal WCAG certification or assistive-technology audit is claimed.

1.4 Site Map

/ (Discovery, Student 1) → /events/:id (Details and registration, Student 1)
/signup and /login (Student 1) → /dashboard (My registrations, Student 1)
Organizer login → /organizer (Studio and statistics, Student 2)
/organizer → /organizer/new and /organizer/events/:id/edit (Student 2)
/organizer → /organizer/events/:id/attendees (Student 2)

Both roles share the header, discovery pages and footer. Navigation exposes the relevant dashboard after login; the backend independently enforces permissions.

The interactive Mermaid sitemap and relational diagram are included in README.md. Responsibility labels describe the assigned demonstration split above.

1.5 Tools and Technologies

Item	Details
HTML version	HTML5 semantic landmarks and forms
CSS framework and version	Tailwind CSS 4.1.16 plus shared responsive CSS [3]
JavaScript (if any)	JavaScript ES modules; React 19.1.1, Vite 7.3.7, Express 5.1.0 [1, 2, 4]
Data source (static / JSON / API)	SQLite through Node 24 node:sqlite; JSON REST API [5]
IDE and Environment used	Codex cloud workspace, Linux, Node 24.19.0, npm 11.9.0, Git
Accessibility Guidelines for Users with Disabilities.	Semantic structure, explicit labels, visible focus, responsive layout; no certification claimed
Testing tools (Lighthouse, WAVE etc.)	ESLint; Node HTTP integration tests; Playwright with Chromium [7]. Lighthouse/WAVE not run.

1.5 Common Design Followed by the Team

A forest-green, sage and warm-neutral palette gives every page the same identity. A shared header contains the brand and role-aware navigation; main content uses restrained headings, cards and forms; the footer identifies the learning project. System typography avoids external font requests. Local SVG illustrations identify event categories. Buttons, badges, tables, loading, empty, success and error states share reusable components. Layouts adapt to mobile without relying on remote assets.

2. Student 1: [STUDENT 1 NAME] (Reg. No: [STUDENT 1 REGISTRATION NUMBER])

Assigned modules: Authentication & Student Dashboard; Event Discovery & Registration.

2.1 Module Description

Authentication & Student Dashboard: signup and login forms validate input, show useful errors, and open a protected personal page. The dashboard displays confirmed/cancelled registrations and permits cancellation before an event starts. Event Discovery & Registration: responsive event cards load from SQLite through the API; search and category buttons narrow the results. Details show the host, date, local time, venue and remaining seats, with a working registration action.

Input is a form, search value or button action. Processing occurs in Express with server validation and database access. Output is JSON used to update the React page and display a confirmation, changed badge, availability, or useful error [1, 4].

2.2 Design of the module

POST /api/auth/register accepts {name,email,password}; the server normalizes email, validates lengths, hashes with bcrypt cost 12 and inserts a unique student account. POST /api/auth/login compares the password hash; both establish an eight-hour opaque session in an HttpOnly, SameSite=Lax cookie. POST /api/auth/logout invalidates it. GET /api/events accepts search/category parameters; GET /api/events/:id returns details and only the current user’s registration. POST /api/events/:id/register runs duplicate, date, open-status and seat checks inside BEGIN IMMEDIATE before inserting or reactivating one registration. GET /api/registrations filters by session user; DELETE /api/registrations/:id checks both user ownership and event time. JSON errors use appropriate 400/401/403/404/409 statuses. HTML constraints assist the user; the server repeats all rules [5, 6].

2.3 Development of the module

Actual password hashing excerpt — server/app.js:

```
const hash = await bcrypt.hash(input.password, 12);
```

Actual booking checks inside transaction(db, ...) — server/app.js:

```
requireValue(existing?.status !== 'CONFIRMED', 409, 'You are already registered for this
  event.');
requireValue(event.status === 'OPEN' && Date.parse(event.starts_at) > Date.now(), 409,
  'Registration for this event is closed.');
requireValue(event.registration_count < event.capacity, 409, 'This event is full.');
```

2.4 Results

The real Chromium walkthrough verified discovery filters and empty states, account creation, rejected credentials, booking confirmation, changed availability, persistent dashboard status and cancellation. API checks also reject duplicates, full/closed/past events and another student’s cancellation. Figures 1–3 show the running application with fictional development data.

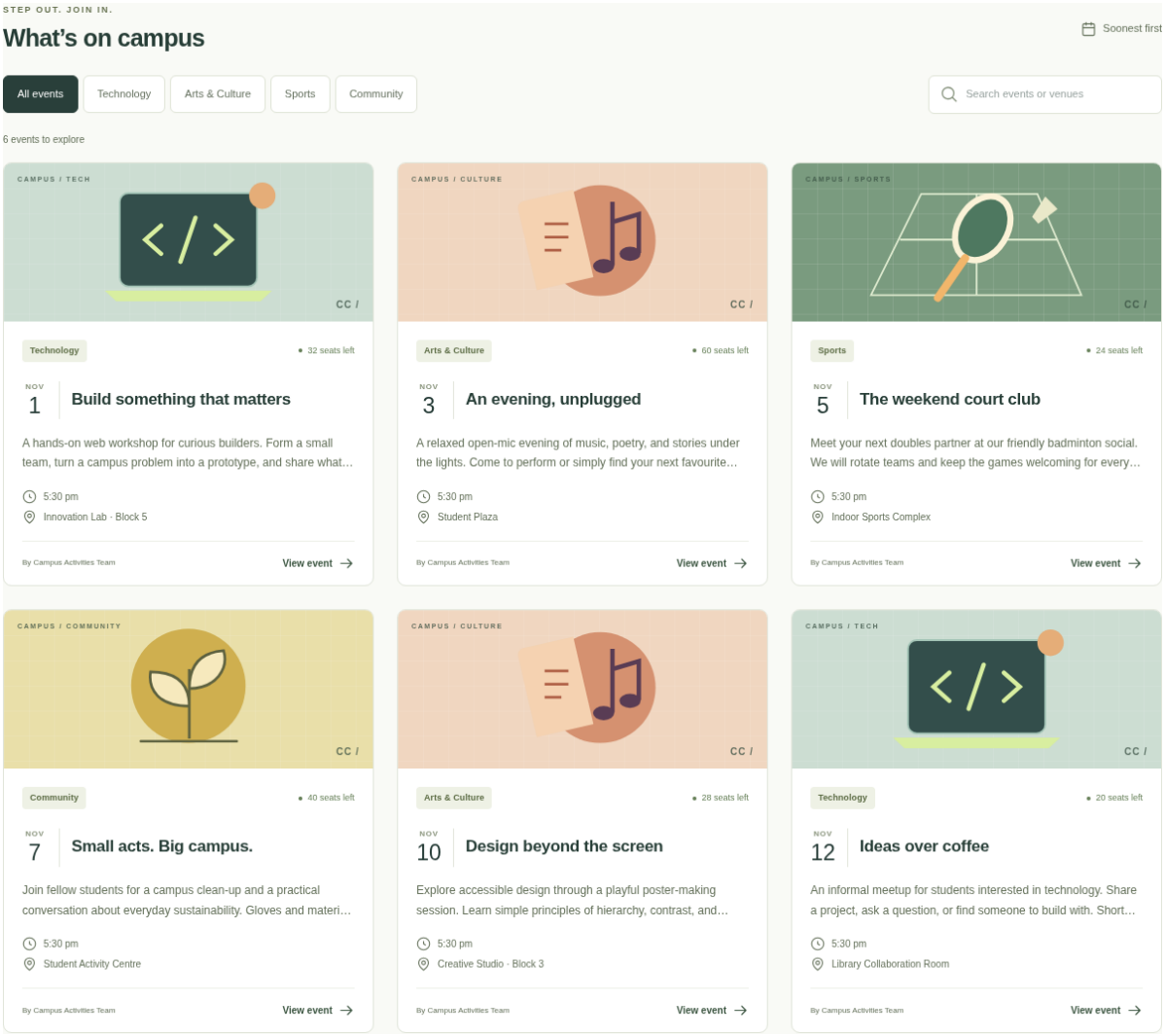


Figure 1. Database-driven discovery with category filters and search.

Source: real Chromium capture; full-resolution original in docs/screenshots/01b-events.png

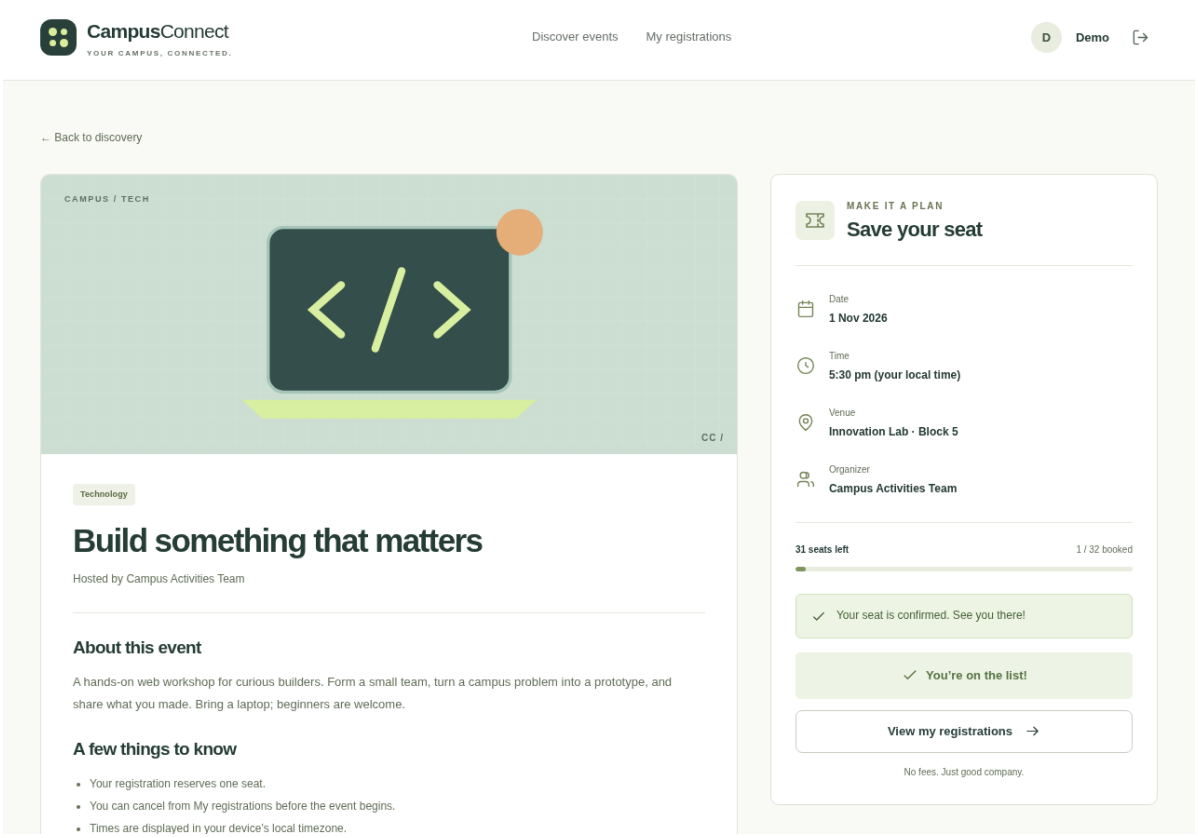


Figure 2. Event details after a successful student registration; availability is updated.

Source: real Chromium capture; full-resolution original in docs/screenshots/02-event-registration.png

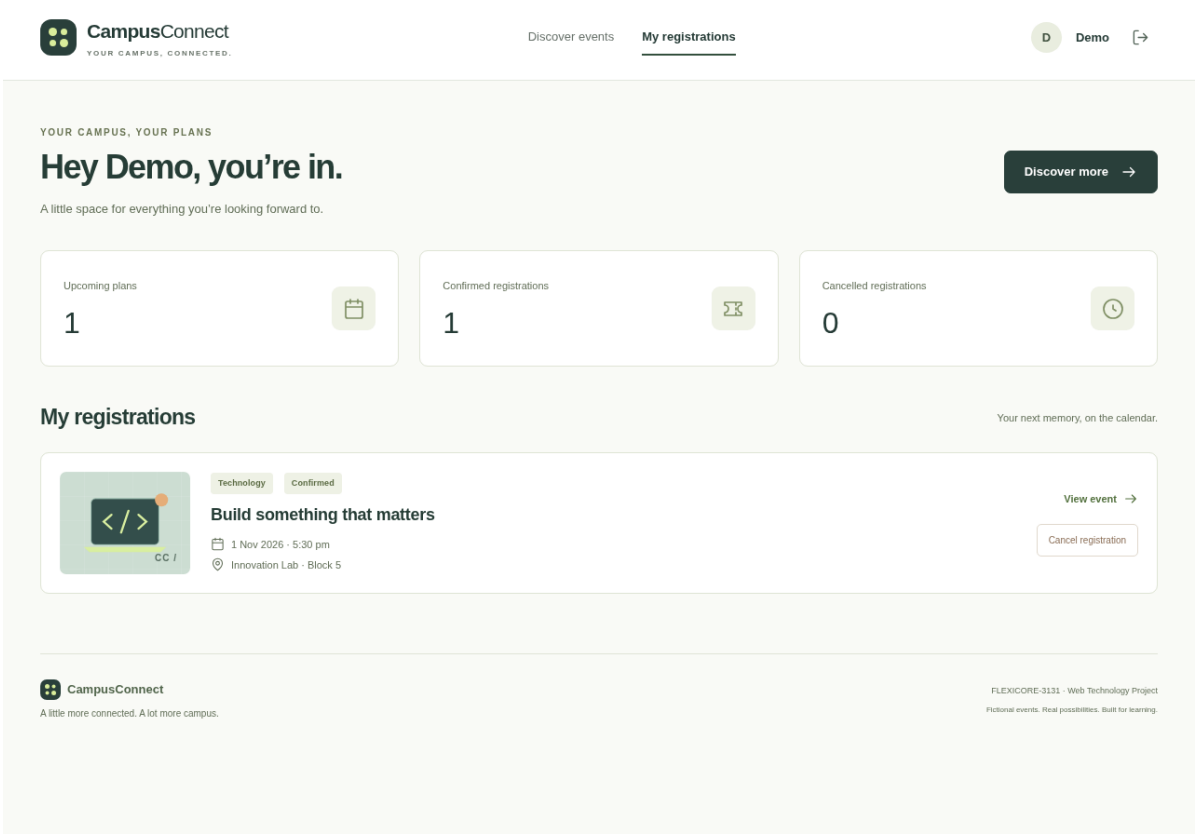


Figure 3. The student dashboard displays a persisted confirmed registration.

Source: real Chromium capture; full-resolution original in docs/screenshots/03-student-dashboard.png

3. Student 2: [STUDENT 2 NAME] (Reg. No: [STUDENT 2 REGISTRATION NUMBER])

Assigned modules: Organizer Event Management; Organizer Dashboard & Attendee Management.

3.1 Module Description

Organizer Event Management: a protected form creates or edits a title, category, description, future date/time, venue, capacity and registration status. The studio allows deletion with confirmation. Organizer Dashboard & Attendees: summary cards display owned-event totals, confirmed bookings and upcoming open events. Each event links to its private list of confirmed students and their registration dates.

Input is an event form or management action. Express checks role and ownership, validates the payload and reads or writes SQLite. The output refreshes the studio or displays actionable validation feedback. Attendee data is never exposed by public discovery routes.

3.2 Design of the module

GET /api/organizer/events scopes results to the session organizer. POST creates an event with the server-supplied organizer ID. PUT /api/organizer/events/:id requires ownership and validates future dates, category, status, text lengths and integer capacity 1–1000; a transaction prevents capacity falling below confirmed registrations. DELETE checks ownership and cascades associated registrations. GET /api/organizer/events/:id/attendees returns only confirmed students for an owned event. Anonymous requests receive 401; student roles and unrelated organizers receive 403. React sends JSON and displays the returned database results. Summary counts are derived from this owned-event data, not hardcoded [4, 5].

3.3 Development of the module

Actual ownership guard — server/app.js:

```
requireValue(event.organizer_id === req.user.id, 403, 'You can only manage your own
  events.');
```

Actual capacity guard inside the update transaction — server/app.js:

```
requireValue(e.capacity >= event.registration_count, 409, 'Capacity cannot be lower than
  confirmed registrations.');
```

3.4 Results

Chromium verified event creation, edit persistence, closing registrations, deletion and the confirmed-attendee list. API tests verified another organizer cannot edit/delete the event or inspect attendees, and students cannot use organizer endpoints. Figures 4–6 show the genuine organizer interface.

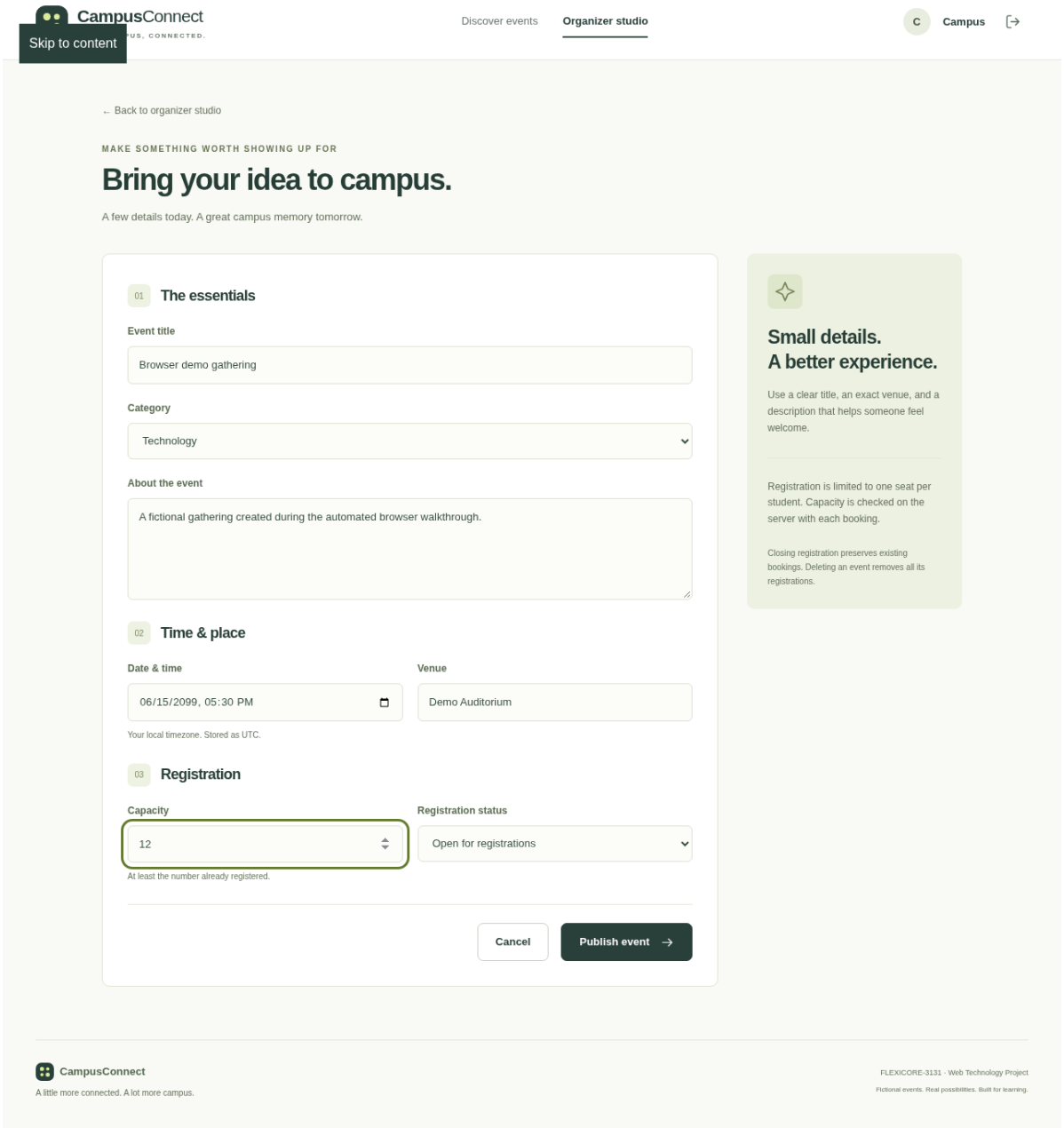


Figure 4. A real organizer event form completed during the browser walkthrough.

Source: real Chromium capture; full-resolution original in docs/screenshots/06-event-management.png

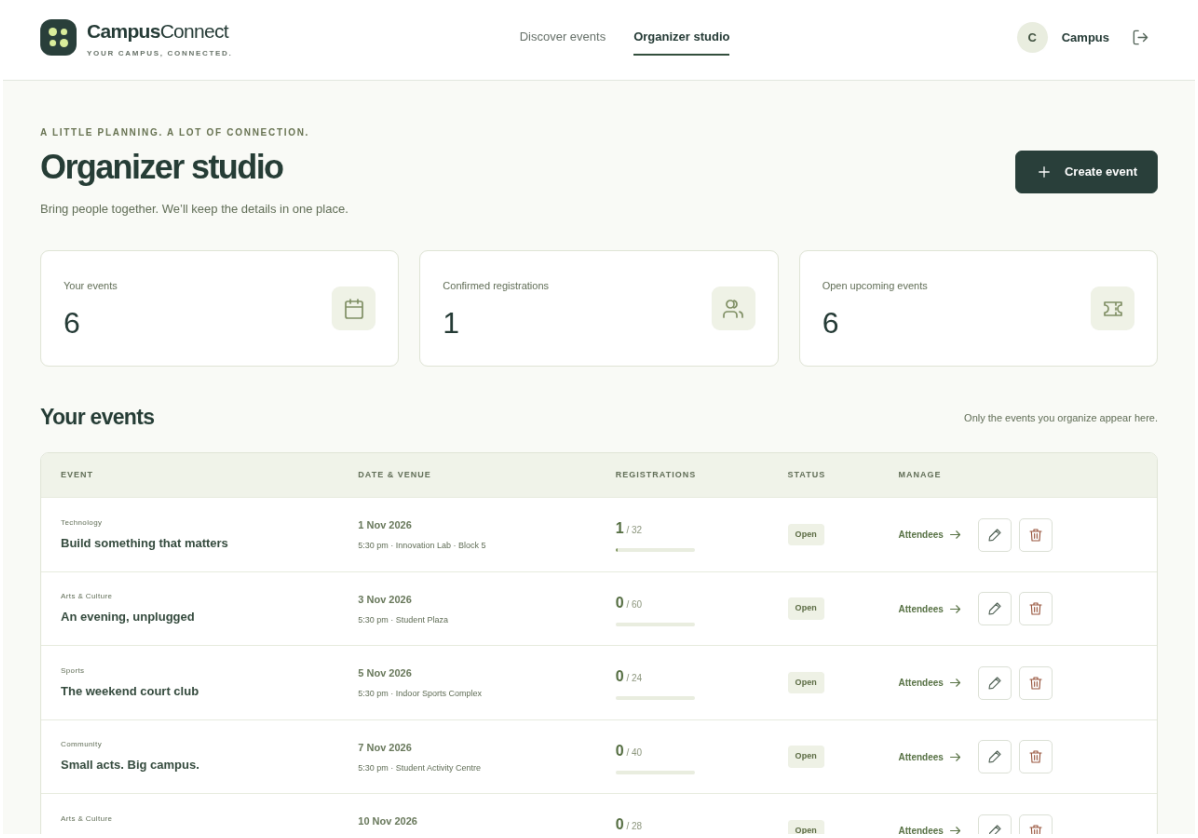


Figure 5. Organizer studio with event totals and confirmed registration counts.

Source: real Chromium capture; full-resolution original in docs/screenshots/04-organizer-studio.png

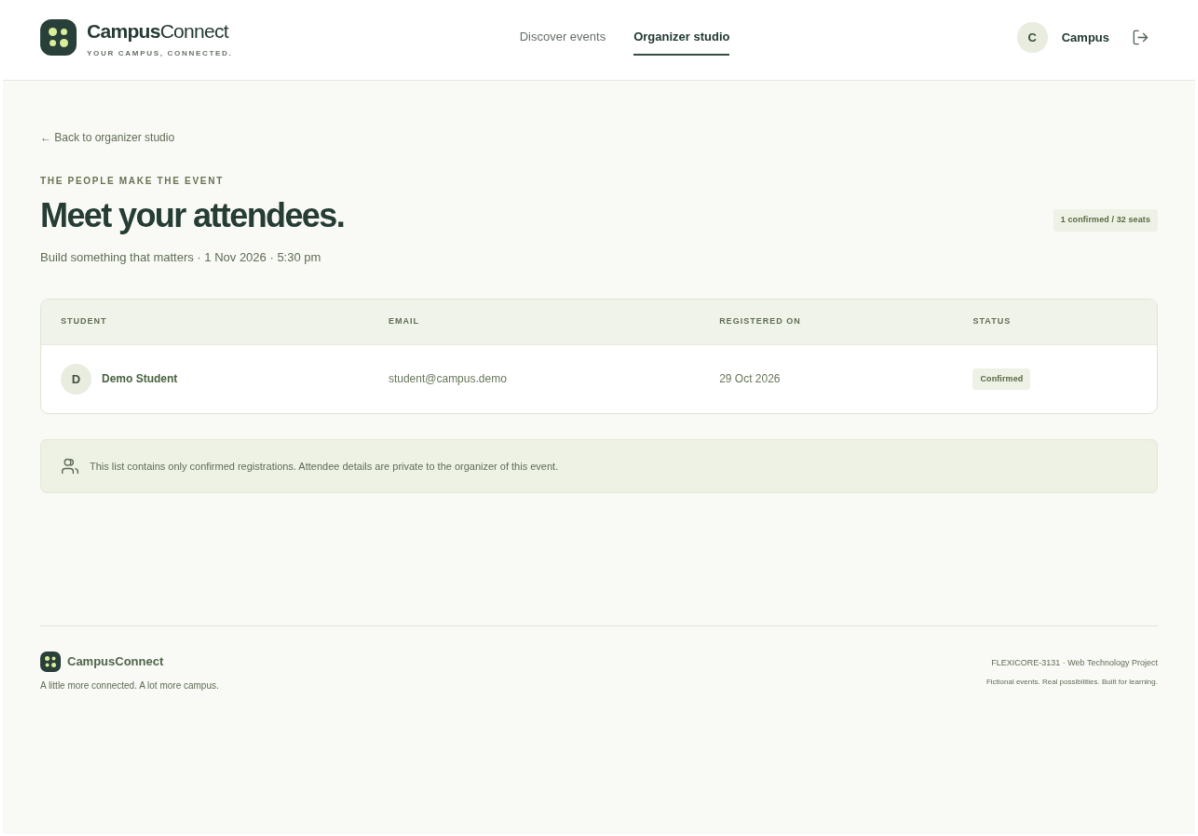


Figure 6. Private confirmed-attendee list using a fictional demo student.

Source: real Chromium capture; full-resolution original in docs/screenshots/05-attendees.png

4. Conclusion and Future Scope

CampusConnect implements six connected features in a small, explainable application.
React forms and views communicate with an Express API using JSON.
SQLite persists users, events and registration history with relational constraints.
Server validation, bcrypt, session cookies and ownership checks protect the core workflows.
Capacity transactions prevent last-seat overbooking and support reliable cancellation.
Focused API and Chromium checks verify the essential classroom journeys.
The design remains responsive and uses local assets without paid services.
Future work could add accessible password recovery and deployment-specific persistent sessions after this assignment.

Verification and limitations

Actual checks: ESLint passed; all 20 HTTP integration tests passed (0 failed, 0 skipped); Vite production build passed; all 10 real Chromium walkthrough steps passed; npm audit reported zero advisories after dependency updates. The browser walkthrough found no unexpected console errors or server exceptions. Screenshots were captured from a disposable SQLite database on 29 October 2026, using Asia/Kolkata for display [7]. A clean-install smoke check is recorded separately in docs/TESTING.md.

Sessions are in memory and end on server restart; database data persists. The fixture password is public and intended only for local demos. No Lighthouse/WAVE scores, full accessibility audit, distributed load test, independent security audit or public deployment are claimed. The PDF is rendered from the completed Word content with ReportLab; its pagination differs from Word.

Two-student demonstration checklist

Student 1: (1) show signup and rejected/correct login; (2) search/filter events; (3) book an available event and show updated seats; (4) show the personal dashboard and cancellation; (5) re-register so Student 2 can demonstrate attendees.

Student 2: (1) log in as the demo organizer; (2) create and edit a future event; (3) show owned-event statistics and a confirmed attendee; (4) close registration and explain server authorization; (5) delete only the disposable demo event. Full steps and expected outcomes are in docs/DEMO.md.

5. References

Official sources consulted through the allowed Node.js and GitHub documentation routes (29 October 2026). Reference numbers are cited in the relevant sections. Main documentation sites restricted by the environment were not represented as successfully consulted.

[1] React, Thinking in React.
https://github.com/reactjs/react.dev/blob/main/src/content/learn/thinking-in-react.md

[2] Vite, Getting Started.
https://github.com/vitejs/vite/blob/main/docs/guide/index.md

[3] Tailwind CSS, official README.
https://github.com/tailwindlabs/tailwindcss/blob/main/README.md

[4] Express, official README.
https://github.com/expressjs/express/blob/master/Readme.md

[5] Node.js 24, SQLite API.
https://nodejs.org/docs/latest-v24.x/api/sqlite.html

[6] bcrypt, official README.
https://github.com/kelektiv/node.bcrypt.js/blob/master/README.md

[7] Playwright, Writing tests.
https://github.com/microsoft/playwright/blob/main/docs/src/writing-tests-js.md